Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

May 12, 2020 · 8 min read · ✨ · Listen

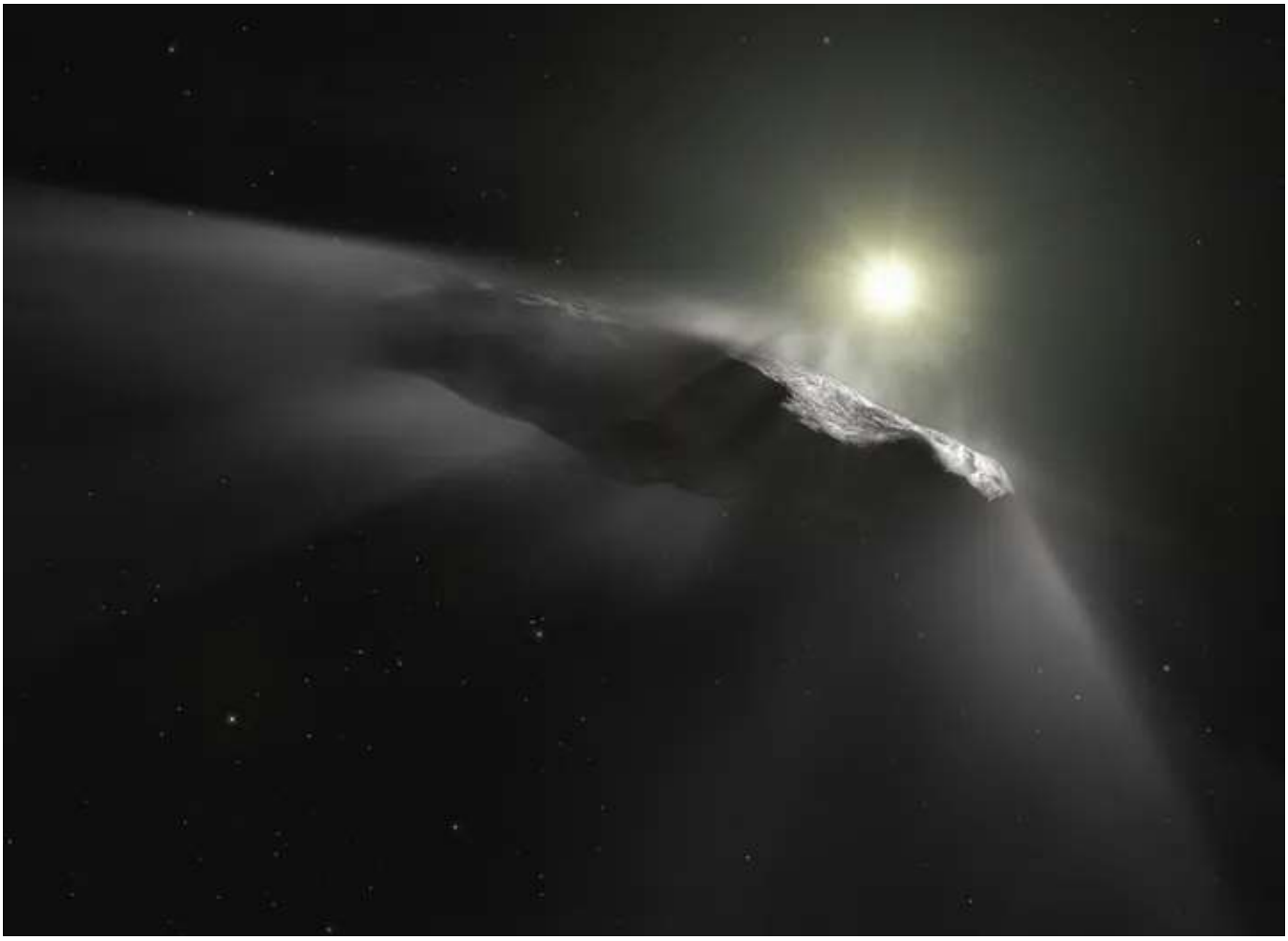
Save

SPACE SCIENCE WITH PYTHON

Space Science with Python: Comets — Visitors from afar

Part 7 of the tutorial series "Space Science with Python" will start a small science project where we will get familiar with icy cosmic visitors: comets

18



Artist impression of the comet `Oumuamua, an interstellar visitor from another star system. Credit: ESA/Hubble, NASA, ESO, M. Kornmesser; License: Creative Commons Attribution 4.0 International License

Preface

This is the 7th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Introduction

Each generation has its "Great Comet". A comet that is visible from Earth, with the naked eye, shining for weeks or months in the night sky with its blurry appearance and long tails. For me, it was the comet Hale-Bopp in the late 1990s. My parents bought me a telescope and from the balcony of our apartment a lifelong passion was born.

Comets are often referred as "dirty ice-balls" from far away. The solid "ball" is the so-called nucleus. If such a nucleus approaches the Sun it heats up causing an out-

gassing of gas and dust particles. Gas and dust are emitted in huge amounts creating three structures around the core part of the comet. These three parts are:

- **The Coma**: A sphere-shaped atmosphere like structure around the nucleus that contains the gas and dust / ice particles
- **The Dust Tail**: Dust particles are distributed along the comet's orbit / path
- **The Ion Tail**: Due to the Sun's radiation the gas particles are charged and are strongly affected by the Sun's magnetic field and the Solar Wind (charged particles that are emitted from the Sun). The ion tail points radially away from the Sun

Besides water and minerals, we know that comets contain also amino-acids and other essential life building blocks. These scientific insights are only possible by sending out robotic spacecraft to these cosmic wanderers. Outstanding missions were:

- **Giotto**: An ESA mission to the comet Halley in the late 1980
- **Stardust**: A NASA mission that collected dust particles 15 years ago from the tail of the comet 81P / Wild 2 and sent them back to Earth
- **Rosetta**: An ESA mission to the comet 67P/Churyumov–Gerasimenko. After a 10 year journey, the spacecraft reached the comet in 2014 and a lander called Philae was deployed

But where do comets come from? Why do some of them appear more frequently? Are they a threat or do we know all of them? (*)

In this and the following next tutorial sessions we will have a closer look at the data. We will determine the comet's origin and compute some essential properties to answer these questions.

Modern space science and astrophysical research is closely related to Data Science and Machine Learning. But as any data scientific work, we need to start with the data engineering part where we collect the data and prepare them for our future work.

Minor Planet Center

Before we start with the data we need to take a look at the libraries we are using today. Like in our last sessions, we need libraries like *datetime* (for date-time parsing), *numpy* and *pandas* for data preparation and engineering. We use *pathlib* later to create a sub-directory and we load *spicepy* to use NASA's SPICE toolkit for the space science tasks.

Last time we created a function that allows us to download files from the Internet (e.g., SPICE kernel files) and store them in a local directory. The function is stored in a new main folder called *_auxiliary*. For now, we only have one common function in there and do not need to create a separate Python library for a single task. Thus, with the package *sys* and the function *sys.path.insert* (line 12 and line 13) we add the scripts and content from the *_auxiliary* folder (line 14).

Part 1/10

A detailed list of known comets is provided by the Minor Planet Center (MPC). The MPC collects all information of smaller bodies populating our Solar System. We download the data that are provided in a compressed JSON format and store them in a folder called *raw_data*. (line 3 + 4 define the folder and MPC URL; line 7 calls the download function that creates the sub-directory and stores the file *cometels.json.gz*). The MPC provides a detailed description of the file's content. We need the information in a few moments.

Part 2/10

For testing purposes we will compute the state vector of a comet in the next section. Thus a SPICE kernel meta file is loaded …

Part 3/10

… with the paths to the *gm_de431.tpc* (contains miscellaneous planetary constants) and *naif0012.tls* (contains time and leap seconds information) kernels.

```
\begindata

KERNELS_TO_LOAD = (
 '../_kernels/pck/gm_de431.tpc'
```

```
    '../_kernels/lsk/naif0012.tls',
    )
```

Now, the compressed JSON file is loaded. The *pandas* function *read_json* reads the compressed file directly and creates a dataframe.

Part 4/10

Let's have a look at the provided data. The file contains orbital elements, the comets' names, references etc. And: 2 different date times; according to the documentation:

```
Columns   F77    Use

[...]
   15 -  18  i4     Year of perihelion passage
   20 -  21  i2     Month of perihelion passage
   23 -  29  f7.4   Day of perihelion passage (TT)
[...]
   82 -  85  i4     Year of epoch for perturbed solutions
   86 -  87  i2     Month of epoch for perturbed solutions
   88 -  89  i2     Day of epoch for perturbed solutions
```

Based on the orbital elements we want to compute the state vector. For this calculation, we will apply the SPICE function *conics*. The function requires the mean anomaly of an object with the corresponding Ephemeris time, called epoch. The mean anomaly is an angle that uses the perihelion direction as a reference direction. Thus, the “Year”, “Month” and “Day of perihelion passage” are the needed date-times. And the mean anomaly that corresponds to the date-time of the perihelion’s passage is (by definition) 0 degrees!

Let’s create a UTC timestamp based on the three date columns (line 10 to 14). A lambda function is applied that merges the year, month and day strings to a date with the format *YEAR-MONTH-DAY*. The Day is given as a decimal number. E.g., 15.5 corresponds to the 15th day 12:00:00 UTC. The “decimal time” needs to be converted to a proper time format. First (line 18), we define a place holder date that can be used with the *datetime.timedelta* function in the lines 23 to 26. The decimal day and time is added to the placeholder date (line 25) and the resulting date-time object is converted to *HOURS:MINUTES:SECONDS* using the *strftime* function (line 26).

Now, the date and the time are merged in line 29 to 32 and the corresponding Ephemeris time (ET) can be computed in line 35 using the SPICE function *utc2et*.

Part 5/10#

We are now ready to compute a state vector for testing purposes, to see, whether we handled the data correctly.

The Great Hale-Bopp

An image of the Great Comet C/1995 O1 (Hale-Bopp). The comet was visible for over a year at the end of the last century. The blue tail is the so called ion-tail and is mostly affected by the Solar Wind. The grey / white

tail consists of dust and ice particles that are distributed along the comet's orbit. Credit: ESO/E. Slawik; License: Creative Commons Attribution 4.0 International License

For the test case we choose the comet Hale-Bopp. *I have to admit that my personal preference is the reason for the choice of this particular comet*.

For the *conics* function we need the orbital elements (stored in the dataframe *c_df*) as well as the *G*M* parameter of the Sun. Line 4+5 extract and define the *G*M* constant from the corresponding planetary kernel using the SPICE function *bodvcd*. Line 8 creates a Hale-Bopp dataframe. The code part …

```
c_df['Designation_and_name'].str.contains('Hale-Bopp')
```

… uses the *Designation_and_name* column and searches for a sub-string that contains *Hale-Bopp*. Via the *loc* function the corresponding Hale-Bopp dataframe row is assigned.

Line 13 to 21 define an input array that contains all data in the correct format for the *conics* function (distances in km, angles in radians, mean anomaly equals 0 degrees).

Part 6/10

The state vector of the comet is computed for the 5th May 2020 (midnight) UTC (line 2+3). We use the NASA HORIZONS web-interface to compute a reference state vector to compare with (**):

Part 7/10

The results are shown below. It can be seen that the results, based on the MPC orbit data, agree well with the NASA HORIZONS data.

```
Comparison of the computed state
vector with the NASA HORIZONS results
```

```
==========================================
X in km (Comp): 5.347972e+08
X in km (NASA): 5.348377806424425E+08
==========================================
Y in km (Comp): -2.702078e+09
Y in km (NASA): -2.702225247057124E+09
==========================================
Z in km (Comp): -5.904119e+09
Z in km (NASA): -5.904425343521862E+09
==========================================
VX in km/s (Comp): 6.856500e-01
VX in km/s (NASA): 6.857065492623227E-01
==========================================
VY in km/s (Comp): -3.265125e+00
VY in km/s (NASA): -3.265390887669909E+00
==========================================
VZ in km/s (Comp): -4.968756e+00
VZ in km/s (NASA): -3.265390887669909E+00
```

Database for science

We downloaded the comet data from the MPC. We checked the consistency of the data by comparing a computed state vector with results from the NASA HORIZONS system. So, we have a solid data fundament for further comet research!

Instead of re-loading the dataframes and parsing the JSON file we create an SQLite database. SQLite allows one to generate a fully functional SQL database that is stored in one single file. This embedded solution is used e.g., on mobile devices. Even larger datasets can be stored in this embedded solution without the need of a dedicated server or distributed file system (*anecdote: for my master thesis and later for my doctorate studies, I used SQLite all the time. I created and worked with SQLite databases that had a size of around 250 GB and surprisingly, they had a good performance*).

But first, let's add 2 more parameters we can derive easily from the orbital elements: the semi-major axis (line 2 to 5) and the aphelion (line 8 to 11). We compute the values only for closed orbits with an eccentricity smaller than 1.

Part 8/10

In the tutorials' main folder a new directory is created for the (comets) database (line 3). Line 6 connects (respectively creates) an SQLite database and a cursor is set in line 7. For now, a single table is created called *comets_main* (line 11 to 25). The primary key is the comet's designation name. Integers or categorical primary keys are better and faster to e.g., create a join between different tables. In our case, we have only a few hundred rows and only a few columns. The performance will be good enough for our purpose. Further, everything stays "human readable". Line 20 adds the mean anomaly and sets a default value of 0 degrees, since the mean anomaly is not provided by the MPC data (only the perihelion passage time).

Part 9/10

We use now the command *executemany* to write the data in the database. We are now ready for further scientific work!

Part 10/10

Conclusion & Outlook

Today we started our cometary science journey. We downloaded data from the Minor Planet Center, analysed the data and used the Hale-Bopp data to verify the content. In the next tutorials our analysis will be based on the created SQLite

database. Data Engineering, preparation and cleaning are the key elements of Data Science as well as for any other scientific field.

Next time, we will get familiar with the data and derive scientific insights from the 800 comets (*e.g., what are the C or P orbit types?*). We will learn some basic SQL commands. If you would like to have a look at the database, I suggest an SQLite viewer like the SQLiteBrowser. Since the database is rather small it is also uploaded on GitHub.

Thomas

Footnotes

(*) Comets, Asteroids, Meteoroids, Planets, Dwarf Planets, and so on … *what is the difference between these objects? Is there even a difference?* Besides the "Space Science for Python" series I would like to share some Space Science knowledge with you in some "Supplementary Space Science Articles".

(**) NASA HORIZONS is only one tool of many other great tools. Whether as a web-interface or desktop application: the "Supplementary Space Science Articles" could also cover additional information on useful tools and programs.

Please let me know if some short Supplementary Space Science Articles would be appreciated.

Python Science Space Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play